

ORIGAMI AS A SPATIAL AND GEOMETRIC REASONING BENCHMARK

Anonymous authors

Paper under double-blind review

ABSTRACT

Multimodal language models perform well on visual recognition and on tool use, but these abilities do not transfer directly to spatial and geometric reasoning. Geometric structure does not reduce to text or to a single image: exact angles and incidences, which surface lies in front of which, and the order in which operations were applied are not recoverable from a semantic description of a scene. We introduce **CP2Seq**, a benchmark for evaluating geometric reasoning and sequential planning in multimodal language models through origami. The benchmark contains 600 procedurally generated samples, each requiring a model to produce a valid sequence of folds from a crease pattern and a target folded state. Samples are produced by a deterministic folding engine. Ground truth is recorded as each fold happens rather than annotated afterwards. We release the generation and evaluation framework to support additional samples and experiments across reference depths and action models. During evaluation, the engine executes proposed folds or rejects invalid actions with explicit feedback, while the model remains responsible for selecting actions and searching for a solution. We evaluate multimodal language models under progressively greater tool assistance, and against a deterministic breadth-first search over the same action set. Candidate sequences are assessed by executing them and comparing the resulting folded state with the target, allowing for planar translations, rotations, and reflections. Under the low reasoning effort setting, none of the evaluated models solves a sample outside the easy subset. On that subset, the best model achieves a success rate of 20.4%, compared with 54.7% for the search baseline. Its dominant failure mode is not an exhausted query budget but the repeated proposal of states it has already visited, which terminates 48% of its attempts. These results identify a substantial gap between multimodal language models and explicit search on this origami planning task.

1 INTRODUCTION

Geometric planning requires a model to connect a description of a target with the operations needed to construct it. In origami, this involves choosing a fold line, identifying which parts of the sheet should move, and tracking how the resulting reflections change their positions and layer order. A locally valid fold may lead away from the target, so completing a sequence also requires selecting among alternatives and revising earlier decisions. We study whether multimodal language models can use visual and geometric observations to make these decisions over successive folds.

Existing origami benchmarks assess some aspects of geometric understanding, but answering questions about a folded configuration does not establish the ability to construct it. GamiBench (Spencer et al., 2025) uses visual question answering to assess folded configurations, viewpoint consistency, and impossible patterns. OrigamiSpace (Xu et al., 2025) includes spatial reasoning tasks and crease-pattern code generation with an interactive environment. Generating a pattern specification differs from constructing a sequence of folds that realizes a supplied target. CP2Seq evaluates whether a model can translate geometric information into a complete, executable folding sequence, with each action checked for validity and the resulting construction checked against the target. Given a crease pattern and target folded state, the model must select folds, track their effects, and revise its choices until execution reproduces both the crease pattern and the target state.

We introduce **CP2Seq**, a benchmark of 600 procedurally generated origami samples and a framework for generating, executing, and evaluating folding sequences. Each task supplies a crease pattern and a target folded state; the reference sequence and intermediate reference states are withheld. Starting from an unfolded square, the model must produce a valid sequence of simple folds under the specified action model. In the visual setting, it receives images of both inputs and views of its current folded state. The crease pattern constrains which creases must be produced, while the target specifies the arrangement of the folded sheet, including layer order. Together, these inputs guide the model’s choice of where to fold and which layers to move.

The task combines *geometric reasoning*, *spatial state tracking*, and *sequential planning*. A fold line is specified in the current folded coordinates, whereas the target crease pattern describes the original sheet. Connecting these representations requires tracking reflections and the correspondence between paper regions. Selecting a fold also requires reasoning about overlapping layers and their connectivity. Subsequent decisions depend on the state produced by earlier actions, so increasing sequence depth extends the planning horizon. We evaluate these abilities through the construction of valid folding sequences across increasing reference depths. The environment represents flat-folded states with explicit layer ordering.

We generate examples procedurally by applying legal folds and recording the resulting sequence, crease pattern, and folded state. This supplies reference solutions without manual annotation and lets us vary reference depth and the permitted folding operations. Releasing the generator also enables fresh evaluation instances beyond the published corpus, reducing reliance on fixed examples that future models may encounter during training (Section 3).

We also provide an evaluation framework that presents images of the input crease pattern, the target folded state, and the current state after each accepted edit. These observations let the model compare its progress with the target and choose its next action. Tools support adding and removing folds, inspecting geometry, and returning to a specified earlier step to explore another sequence. The engine checks proposed edits and returns feedback, while the model directs the search. Different levels of tool assistance allow us to study how this support affects performance (Section 4.3).

Evaluation checks both individual actions and the completed construction. Candidate sequences are replayed and assessed for crease-pattern agreement and agreement with the target folded state. A sequence need not match the recorded actions to be correct. Folded-state comparison accounts for a global translation, rotation, or turnover of the completed object, with the corresponding changes to layer order and face orientation (Section 4.2). We also provide a deterministic breadth-first search baseline using the same candidate-fold enumerator and terminal-state comparator. This baseline measures how much can be achieved by systematic search with the available filtering, providing context for model performance beyond comparisons between language models.

Our contributions are:

1. **A dataset and generation framework.** CP2Seq contains 600 samples with crease patterns, target states, recorded fold sequences, and intermediate states. Configurable generation supports experiments across reference depths and action models, with coupling recorded to characterize geometric structure.
2. **An interactive evaluation and verification pipeline.** The framework connects visual and geometric inputs to tools for constructing and revising sequences, step-level legality checks, and final replay-based scoring. It includes corpus checks and a folded-state comparator. It defines four assistance conditions; the current experiments cover the filtered and basic-tools conditions.
3. **A search baseline and an empirical analysis of model behavior.** We provide breadth-first search over the same candidate actions, report performance by difficulty stratum, and analyze episode termination. The current evaluation concentrates on `gpt-5.6-luna`; other model runs have insufficient coverage for a cross-model ranking.

In the current aggregate results, breadth-first search solves 54.7% of easy-stratum attempts, compared with 20.4% for `gpt-5.6-luna` with filtering and 6.2% with basic tools. These rates summarize different sets of attempts rather than a paired comparison on an identical sample set. No evaluated language-model attempt succeeds on the medium or hard strata. All recorded reasoning-effort settings are low, so these results do not establish a general limit on the evaluated models. State

cycling terminates 48.1% of Luna’s attempts, motivating further experiments on state tracking and backtracking. Section 5 reports the coverage, budgets, and limitations of these measurements. The findings motivate evaluating geometric planning through both the constructions models produce and the search behavior that leads to them.

2 RELATED WORK

2.1 ORIGAMI BENCHMARKS AND GENERATION

GamiBench (Spencer et al., 2025) evaluates multimodal models through visual questions about folded configurations, viewpoints, and impossible crease patterns. Its viewpoint-consistency and impossible-fold-selection metrics assess whether predictions remain consistent across views and whether models recognize infeasible patterns. OrigamiSpace (Xu et al., 2025) evaluates pattern prediction, multi-step spatial reasoning, spatial relationships, and crease-pattern code generation. It also provides an interactive environment for the code-generation task. These benchmarks connect origami geometry to visual reasoning and generation, with different requirements on the model’s output.

CP2Seq focuses on recovering a complete folding sequence from a crease pattern and a target folded state. The model selects and revises actions, and evaluation checks their execution and the resulting construction. This differs from selecting a folded configuration or generating a pattern specification. Our procedural generator records reference sequences as folds are performed, allowing new instances to be produced without manually annotating solutions. The environment checks proposed actions under its supported folding model and supplies feedback during the search. Rejecting an invalid action in a reachable instance is distinct from identifying an impossible crease pattern: the former concerns a particular move, while the latter concerns the feasibility of the pattern itself.

2.2 COMPUTATIONAL ORIGAMI AND SEQUENCE PLANNING

Our action models build on the simple-fold framework of Arkin et al. (2004), which distinguishes one-layer, some-layers, and all-layers folds. CP2Seq uses the latter two variants. Work on flat-folded states studies layer ordering as well as planar geometry (Bern & Hayes, 1996; Akitaya et al., 2024), motivating our explicit representation of layers in the state and final comparison. Earlier work also addresses generating folding sequences from crease patterns (Akitaya et al., 2013); sequence recovery is therefore an established computational origami problem. We use it to evaluate multimodal models through visual observations, executable actions, and feedback. Complexity results depend on the permitted folds and crease-pattern restrictions (Arkin et al., 2004); we assess the practical difficulty of our generated instances through measured model and search-baseline performance.

3 CP2SEQ: TASK AND DATASET

3.1 TASK AND ACTION MODELS

Given a crease pattern C and a target folded state S^* , the model must construct a sequence of legal folds $a_1, \dots, a_T$ starting from an unfolded square. Replaying the sequence must reproduce the crease pattern and reach the target state under the equivalence defined in Section 4.2. Each state records paper geometry and layer ordering; the reference sequence and its intermediate states are withheld, and alternative valid sequences are accepted. Figure 1 summarizes the task.

We use two simple-fold action models (Arkin et al., 2004): all-layers folds move the paper on the selected side of a fold line across the full stack, while some-layers folds select a contiguous run from the top or bottom of the stack. Both use flat 180° folds, with connectivity checks to reject moves that would tear the sheet; state representations and action constraints are detailed in Appendix A.

3.2 DATASET CONSTRUCTION

Starting from a square sheet, the generator samples a legal fold under the chosen action model, applies it, and records the action and resulting state. This process continues to the requested reference

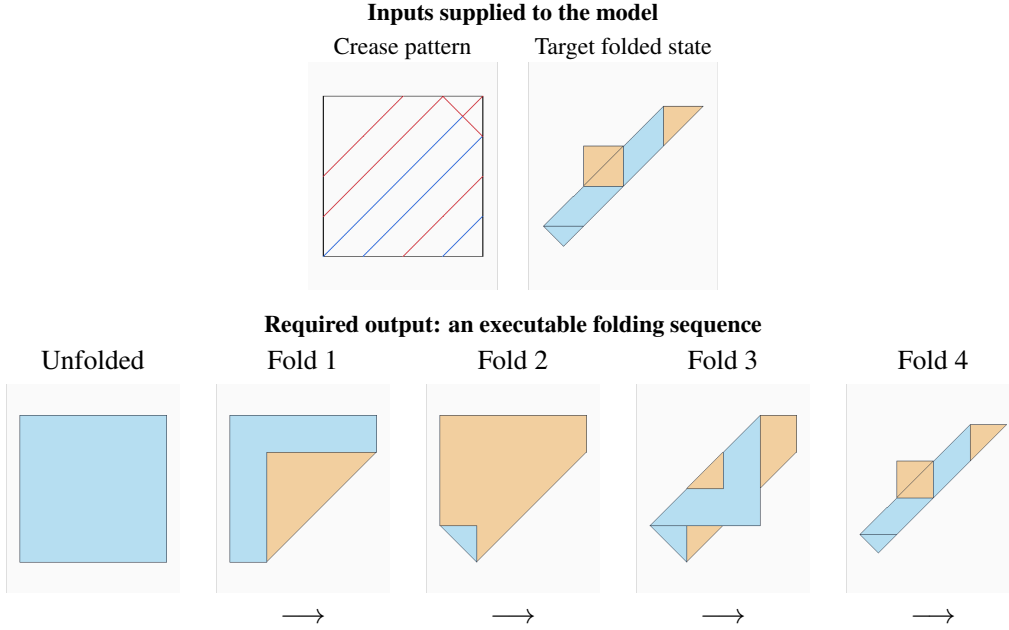


Figure 1: CP2Seq task illustrated by all-layers/easy-0001. The model receives the crease pattern and target state and must produce a valid sequence from the unfolded sheet. The lower row shows states from the recorded reference sequence for illustration, not a model prediction; this sequence and its intermediate states are withheld during evaluation.

depth; attempts with no available legal continuation are discarded. The accumulated creases are mapped to the original sheet to form the input crease pattern, with edges split at intersections. The last state supplies the target, and the recorded actions supply a reference solution. A replay check compares the generated sample with its recorded construction.

We group samples by reference fold count, hypothesizing that longer sequences generally require more search and are therefore harder to recover. These groups describe expected difficulty; reference length need not equal the shortest valid solution length. We also record coupling, the number of creases created by a fold, to characterize the geometry of each instance.

The generation procedure is summarized below.

```

199 GENERATE(reference_depth, action_model):
200     state <- unfolded square
201     actions, frames <- [], []
202     for step in 1..reference_depth:
203         candidates <- LEGAL_FOLDS(state, action_model)
204         if candidates is empty: return FAILED
205         action <- SAMPLE(candidates)
206         state <- APPLY(state, action)
207         actions.append(action)
208         frames.append(state)
209     cp <- CREASE_PATTERN_IN_ORIGINAL_SHEET(state)
210     cp <- SPLIT_EDGES_AT_INTERSECTIONS(cp)
211     if not REPLAY_CHECK(cp, actions, state): return FAILED
212     return (cp, state, actions, frames, METADATA(actions))

```

3.3 DATASET STATISTICS

CP2Seq contains 600 samples: 400 all-layers examples grouped by reference depth and 200 some-layers examples with three to six reference folds (Table 1). These counts describe distinct dataset samples, not evaluation attempts.

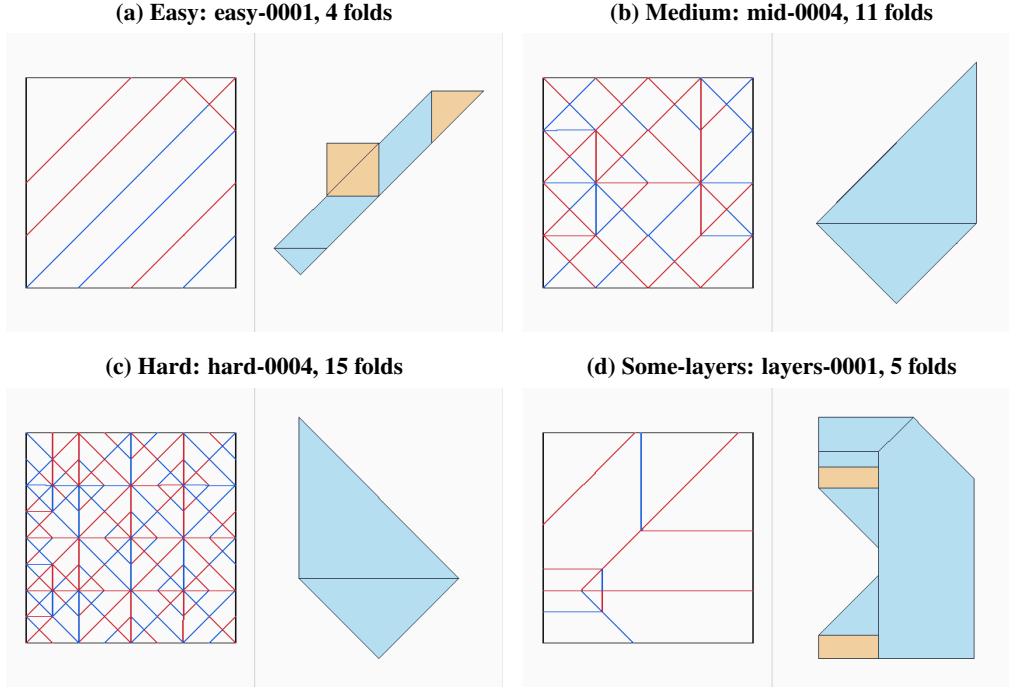


Figure 2: CP2Seq examples: each panel pairs the crease pattern (left) with the target top view (right). Panels (a–c) use all-layers folds; (d) is from some-generated-d5. Counts are reference folds. Red and blue denote mountain and valley creases.

Table 1: Dataset composition. Fold counts refer to recorded reference sequences.

Action model	Stratum or group	Samples	Reference folds
All-layers	Easy	200	4–10
All-layers	Medium	100	11–13
All-layers	Hard	100	14–19
Some-layers	Depth 3	50	3
Some-layers	Depth 4	50	4
Some-layers	Depth 5	50	5
Some-layers	Depth 6	50	6
Total		600	3–19

Each sample contains the task inputs, a reference sequence, and metadata (Table 2). The model receives the crease pattern and target state; reference actions and intermediate reference states are withheld. Tool availability and target feedback are configured separately from dataset groups (Section 4.4.1).

Table 2: Contents of a CP2Seq sample.

File	Contents
cp.fold	Input crease pattern.
steps.fold	Folded states after each reference action; only the final state is supplied as the target.
seq.json	Withheld reference actions: fold lines, directions, layer selections, and creases created.
meta.json	Sample descriptors and generator version.

Reference sequences are checked by replaying them and comparing the resulting creases with the stored pattern. The two corpus checks reported in this release pass on all 600 samples; they share the folding engine and therefore do not independently validate its physical assumptions.

Coverage is limited in two respects: long reference sequences with low coupling are sparsely represented by the generation procedure, and some-layers examples extend only to six reference folds. These are limits of the present corpus, not proofs that other configurations are impossible.

4 EXPERIMENTAL SETUP

4.1 ENVIRONMENT

The model edits a folding sequence through the tools in Table 3. For each proposed fold, the engine reflects the selected paper across the fold line, updates layer order and orientation, and checks sheet connectivity and compatibility with the input crease pattern. Accepted edits update the current state; rejected edits return an error code and preserve the previous state. Candidate enumeration and target-comparison feedback are optional run settings.

For a some-layers fold, moving and stationary regions may share an edge away from the hinge. Reflecting only the moving region would separate that shared edge, so the engine rejects the proposal as `would-tear`. A `no-crease` rejection means that paper was selected to move but the operation did not create a crease segment; it is distinct from the connectivity check. Other rejections cover an empty moving region, an incompatible over/under direction, and creases absent from the target pattern. Detailed conditions are listed in Appendix A.6.

Observations include the current sequence, layer geometry, and rendered views. The image tools provide top, oblique, reverse, X-ray, and exploded views; the exposed views depend on the run configuration. Exploded views use illustrative layer separation. Final evaluation compares the constructed crease pattern and folded state with the supplied targets (Section 4.2.1).

The repository also provides a separate Flat-Folder check for the existence of a flat-folded state consistent with a crease pattern (Akitaya et al., 2024). This check is distinct from action execution and target matching; its scope is described in Appendix A.6.

4.2 EVALUATION METRICS

The recorded sequence provides one solution, but other sequences may produce the required construction. We therefore assess correctness by execution rather than agreement with the reference actions. Distance between the constructed crease pattern and the input crease pattern is a separate measure of geometric progress; it does not compare action sequences or replace final-state verification.

4.2.1 VERIFICATION

We execute the proposed sequence and compare its accumulated creases with the input pattern, including mountain–valley assignments. We then compare the final stack with the target after alignment by a single global translation, rotation, or turnover. A turnover reflects the geometry, reverses the stack order, and flips face orientations. The same transformation must align all layers; independent layer transformations and arbitrary layer permutations are not accepted. A successful construction must satisfy both crease-pattern and folded-state agreement.

The engine’s crease comparator uses a numerical tolerance of 2×10^{-6} for interval endpoint matching, and the shared folded-state comparator uses the same positional tolerance. The comparison is therefore not tolerance-free. Appendix A.5 describes the alignment and representation details. These final checks are distinct from the three optional levels of target feedback supplied during an episode.

4.2.2 REPORTED METRICS

We report solve rate by fold-count stratum, with the number of attempts and the applicable run budgets. Termination reasons distinguish successful submission, unsuccessful completion, budget limits, and repetition-related stopping. Query counts describe search effort; for unsuccessful attempts they are consumed queries, not observed queries to solution.

We measure CP edit distance between the accumulated crease pattern in the last recoverable state and the supplied pattern, in original-sheet coordinates. Collinear mountain and valley intervals are merged separately, excluding sheet boundaries. If m intervals match under the crease comparator, the insertion/deletion distance is $d_{CP} = |C_{target}| + |C_{candidate}| - 2m$. Each missing or extra interval costs one; changing an assignment costs two. Partially overlapping intervals do not match unless their endpoints agree within tolerance. We also report $d_{CP}/|C_{target}|$, which can exceed one. Normalized distance is undefined for an empty target. Means weight attempts equally and exclude missing state artifacts, whose counts are reported separately. This metric compares crease patterns, not fold sequences, and zero distance does not establish folded-state agreement.

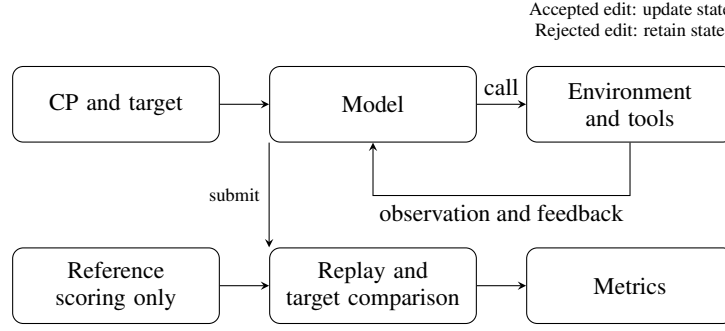


Figure 3: Evaluation pipeline. The model edits and inspects its current construction. The reference sequence is withheld from the model and used only for scoring.

Comparator validation must cover accepted global transformations and rejected changes to layer order or orientation. The validation applies to the implementation used to score each run; checks of a different comparator do not establish the behavior of the reported evaluation.

4.3 EVALUATION PROTOCOL

4.3.1 EPISODE PROCEDURE

One episode is one sample attempted by one model under one tier of assistance. The loop is the same for every arm and every model, and the only thing that varies between arms is what the model is allowed to see and call.

```

EPISODE(sample, model, condition, query_budget):
    cp, target <- LOAD_INPUTS(sample)
    session <- INITIALIZE(cp, target, unfolded_square)
    history <- []
    for query in 1..query_budget:
        obs <- OBSERVE(session, condition)
        call <- model(cp, target, obs, history, condition)
        if call is finish: break
        # Edits: add/remove fold, go to step, restore revision.
        # Reads: state, images, candidates, target feedback.
        result <- DISPATCH_TRANSACTIONALLY(session, call)
        # Rejected edits leave session unchanged but consume a query.
        # Read calls consume a query without changing the sequence.
        history.append(call, result)
        if STOPPING_RULE(session, history): break
    return SCORE(REPLAY(session.actions), REFERENCE_AT_SCORING(sample))
  
```

The pseudocode describes one tool call per query. Accepted edits update the current sequence; rejected edits preserve it, and both consume budget. Removing a fold replays the remaining actions; returning to an earlier step truncates the sequence, while revision restoration can recover a previous branch. The reference sequence is withheld until scoring. Run-specific turn, time, and cycle limits must be reported alongside the query count.

4.3.2 TOOL CONFIGURATION

Table 3 lists the tool interface. Loading and configuration are described in Appendix B.5.

4.3.3 MODELS AND BUDGETS

Conditions. Section 4.4 distinguishes completed configurations from planned comparisons. Matched samples and budgets are required for controlled comparisons; the current aggregate does not establish such matching.

Models. [TO FILL from the aggregation: the models actually run, their versions, and the reasoning-effort and image-history settings used. Held fixed across tiers, because comparing across models and across tiers at once would confound the two.]

Table 3: Tools available to the evaluation harness; exposure depends on run configuration.

Tool	Operation
<code>add_fold</code>	Append a checked fold; reject invalid actions without changing state.
<code>remove_fold</code>	Remove a specified fold and replay the remainder transactionally.
<code>go_to_step</code>	Retain the first specified number of folds.
<code>restore_revision</code>	Restore a previously recorded branch state.
<code>get_state</code>	Return the current sequence and layer geometry.
<code>get_images</code>	Render views of a completed step, including X-ray and exploded views.
<code>list_legal_folds</code>	Optionally enumerate legal candidates compatible with the target CP.
<code>compare_to_target</code>	Optionally return feedback at the configured level (Section 4.4.1).
<code>finish</code>	End the episode and evaluate the current sequence.

Protocol. Report budgets, repeat counts, and stopping rules for each configuration. The current results aggregate attempts; per-sample repeats and their variability have not yet been reported (Section 5.3).

Transforms are applied in every reported result. Every solve rate in Section 5 is computed after quotienting out the plane isometry group of Section 4.2.1: translation, rotation and reflection, with reflections reversing the stack and flipping face parity. A candidate is counted correct if one single isometry carries every layer onto its reference counterpart. This is not a post-hoc leniency applied to borderline cases; it is part of the definition of a correct answer, applied identically to every arm, every model and every baseline. Turning it off would mark correct answers wrong, because where a folded model lands on the table is an accident of which half of the sheet travelled.

Matching under transforms requires a floating-point tolerance, and it is stated. Admitting arbitrary rotation means candidate and reference coordinates agree only to within floating-point representation, so the comparator matches polygons within 2×10^{-6} , cleans degenerate vertices at 10^{-7} and treats vertices as collinear below 10^{-9} . Every result record carries the tolerance it was scored under, so the value is visible in the released runs rather than buried in a constant. Crease compatibility and terminal-state comparison both involve numerical tolerances; these checks must not be described as tolerance-free.

Error verbosity is itself a variable. The richer a refusal, the more of the reasoning the environment performs rather than the model; at the limit a message that names the fix has solved the step. Verbosity is fixed before the first run and reported, otherwise the comparison between tiers measures message design rather than reasoning.

4.3.4 BASELINES

Breadth-first search has recorded results. Uniform random search and the symbolic solver below remain planned baselines; they are not included in the reported comparison.

Table 4: Baseline configurations and status.

BASLINE	WHAT IT ESTABLISHES
Uniform random over enumerated legal actions (planned)	The floor. What the tool alone achieves with no reasoning at all
Deterministic breadth-first over the same action set (workspace/search_baseline.mjs)	How much of the task the enumerator already solves, and the depth at which exhaustive search stops being affordable
Symbolic crease-pattern-to-sequence solver (Creasy) (planned)	The closest thing to prior art on the task itself

The breadth-first baseline is the one that answers the “the simulator did all the work” objection of Section 4.1 numerically rather than structurally, which is the stronger form of that answer.

4.4 ASSISTANCE CONDITIONS

The reported conditions are *basic tools* and *basic tools with candidate filtering*. Basic tools let the model add and remove folds, return to an earlier step or revision, inspect the current state and images, and submit its sequence. Candidate filtering additionally provides `list_legal_folds`. The basic-tools condition does not provide candidate enumeration or the optional target-comparison tool. No-vision and verifier-only comparisons are not reported in the current results.

Table 5: Assistance configurations and current reporting status.

Configuration	Status and interpretation
Candidate filtering	Results reported for <code>legal_folds</code> ; available tools must be recorded per run.
Basic tools	Fold editing, backtracking, state inspection, images, and submission; no candidate enumeration.
No visual feedback	Planned controlled comparison; no results reported.
Verifier only	Planned controlled comparison; no results reported.

A controlled comparison should hold the model, sample set, budgets, and other settings fixed while changing the specified assistance. The saved aggregate does not establish this matching, so its rates describe observed configurations rather than isolated causal effects of individual tools.

4.4.1 TARGET-COMPARISON FEEDBACK

The optional `compare_to_target` tool has three feedback levels, configured for the run rather than selected by the model. These levels are separate from the assistance conditions above and from final verification in Section 4.2. The tool reads the current and target stacks without consulting the reference sequence or changing the state.

Table 6: Implemented target-feedback levels. Their availability does not establish an experimental benefit; comparisons require matched runs.

Level	Information returned
1: Counts	Current and target layer counts, their difference, and a count-based summary.
2: Diagnosis	Level 1 plus aligned layer matches, mismatched ranks, and a diagnosis of count, shape, orientation, or ordering disagreement.
3: Corrections	Level 2 plus suggested corrections for up to twelve mismatched ranks, such as orientation changes or a different target rank for a layer shape.

The richer levels supply more target-specific guidance. Their diagnoses are geometric feedback, not proofs of a valid repair or substitutes for the final state comparator. In particular, a matching layer count does not establish that the sequence is complete or correct.

5 RESULTS

Solve-rate and termination statistics come from `workspace/RESULTS/results.md`, produced by `python3 workspace/aggregate_results.py` over every saved run. 1,211 attempts across 47 runs and 256 distinct samples. Every attempt is counted once, and an attempt with no recorded `solved` field is counted as unsolved.

Crease-pattern distance. The saved-run CP-distance scan covers 1,422 attempts, of which 1,421 have recoverable crease patterns. Mean insertion/deletion distance is 45.454; mean distance normalized by target crease count is 0.553. One attempt is excluded because its state could not be recovered. These averages pool models, baselines, configurations, and repeated samples; they describe the saved attempts rather than a matched comparison of methods. This scan includes runs outside the earlier 1,211-attempt solve-rate snapshot. These attempt-weighted averages pool saved configurations and repeated samples; they are descriptive and do not constitute matched model comparisons. The CP-distance scan may cover more runs than the earlier solve-rate snapshot.

Table 7: CP insertion/deletion distance over saved attempts. Means exclude unscored attempts. Normalization divides each distance by its target merged crease count before averaging.

Model	Attempts	Scored	Mean distance	Mean normalized
claude-sonnet-5	5	5	12.800	0.481
codex-cli-chatgpt	1	1	8.000	1.000
deterministic-bfs	725	725	38.810	0.494
deterministic-reverse	203	203	24.212	0.764
gpt-5.6-luna	447	447	67.159	0.538
gpt-5.6-sol	7	7	15.857	0.586
gpt-5.6-terra	1	1	0.000	0.000
gpt-6-astra	1	1	0.000	0.000
unknown	32	31	43.065	0.804
All attempts	1422	1421	45.454	0.553

Stratum (reference folds)	Median states expanded	Solved / attempts
Easy (4–10)	65	366/669
Medium (11–13)	1,051.5	1/40
Hard (14–19)	532	0/16

Figure 4: Observed BFS search effort in the saved aggregate. Bar lengths use a linear scale and show median states expanded across attempts, including unsuccessful attempts. Budgets and repeated samples are not standardized by this aggregate; 358 of 725 attempts terminated in timeout. The lower hard-stratum median does not imply that hard instances require less search. Reference depth is not minimum solution length.

5.1 MODEL PERFORMANCE

No language model solved a single sample beyond the easy stratum. Across every model arm, mid is 0 of 70 and hard is 0 of 27. On the easy stratum a deterministic breadth-first search over exactly the action set the models are given solves 54.7%, against 20.4% for the best-covered model at the same tier.

Table 8: Solve rate per stratum, all saved runs. Denominators are attempts, not distinct samples. Rows with fewer than ten attempts support no rate and are shown only so that reporting is not selective.

ARM	TIER	EASY	MID	HARD	OVERALL	CALLS
deterministic-bfs	legal-folds	366/669 (54.7%)	1/40 (2.5%)	0/16	367/725 (50.6%)	5
gpt-5.6-luna	legal-folds	59/289 (20.4%)	0/66	0/25	59/380 (15.5%)	23.5
gpt-5.6-luna	basic tools	4/65 (6.2%)	0/1	0/1	4/67 (6.0%)	29
claude-sonnet-5	legal-folds	0/5	–	–	0/5	10
gpt-5.6-sol	basic tools	0/7	–	–	0/7	20
gpt-5.6-terra	basic tools	0/1	–	–	0/1	35
gpt-6-astra	basic tools	1/1	–	–	1/1	5
codex-cli-chatgpt	none	0/1	–	–	0/1	2
errored, model unrecorded	–	0/20	0/3	0/1	0/24	–

Caveat: read the denominators before the percentages. Only gpt-5.6-luna and the baseline have coverage worth a rate. gpt-6-astra shows 1/1; that is one attempt and must not be reported as 100%. claude-sonnet-5, gpt-5.6-sol, gpt-5.6-terra and codex-cli-chatgpt have between one and seven attempts each. These rows are present because omitting them would be selective reporting, not because they support a comparison. Any main table in the submitted paper prints n in every cell or drops the row.

5.2 SEARCH BASELINE

deterministic-bfs solves 367 of 725 attempts overall. Per stratum: 366/669 easy, 1/40 mid, 0/16 hard. Median states expanded is 65 on easy, 1051.5 on mid, 532 on hard. 358 of its 725 attempts ended in timeout rather than exhaustion, so **50.6% is a lower bound under the baseline’s time budget, not the ceiling of exhaustive search.**

Candidate enumeration filters for legality and compatibility with the target crease pattern. The baseline measures performance of systematic exploration with this filtering; its success rate does not by itself identify which reasoning abilities a model uses.

On easy instances, Luna solves 59/289 attempts (20.4%) with candidate filtering and 4/65 (6.2%) with basic tools. These are descriptive rates from unmatched attempt sets; controlled comparisons and the remaining assistance experiments are pending.

5.3 FAILURE ANALYSIS

Table 9: Termination reasons for gpt-5.6-luna, 447 attempts.

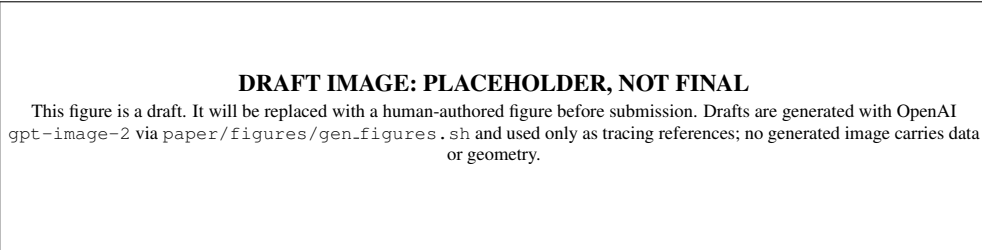
TERMINATION	<i>n</i>	SHARE
state_cycling	215	48.1%
finished	167	37.4%
turn_budget	47	10.5%
repetition_detected	18	4.0%

State cycling terminates 215 of 447 Luna attempts (48.1%), while another 18 terminate with a repetition label and 47 reach the turn budget. These labels describe stopping behavior rather than proving the absence of reasoning. Repeated exploration without reaching the target indicates ineffective search in those attempts; attributing it to state tracking or action selection requires trajectory analysis. A `finished` label denotes submission, not necessarily success.

The harness defaults include a limit of three repetitions of the same action from the same position and fifteen arrivals at the same position by any route. The latter is a revisit limit, not a limit of fifteen distinct branches. Such limits can stop repeated exploration and must be reported from each run’s configuration rather than inferred from defaults. The aggregate table does not separately identify all revisit-limit events.

No model attempt solves a medium or hard instance in the reported aggregate; BFS solves one medium instance. This observation is consistent with greater search difficulty at larger reference depths, but does not establish a causal effect of depth or coupling.

The aggregate contains 725 BFS attempts and 447 Luna attempts, with much smaller counts for other models. Recorded reasoning settings are low for 461 model attempts and absent for 25. Twenty-four attempts with missing model identity ended in error and are reported separately as unsolved. Per-sample repeat statistics and matched-budget comparisons remain to be reported.



Placeholder: solve rate by fold-count stratum for Luna and BFS, with attempt counts and run conditions shown.

Figure 5: [DRAFT IMAGE] Planned summary of performance by fold-count stratum.

6 LIMITATIONS

The current evaluation is concentrated on Luna and the search baseline; other model runs have limited coverage. All 461 model attempts with a recorded reasoning setting used low effort, while 25 lack that setting. The aggregate contains repeated attempts and does not establish matched samples and budgets across configurations. Higher-effort runs and controlled assistance comparisons remain to be completed.

Fold-count groups express the hypothesis that longer reference sequences generally require more search. They do not establish minimum solution length or guarantee an ordering of instance difficulty. The action space covers all-layers and selected some-layers simple folds, excluding pre-creasing; the some-layers corpus currently extends to six reference folds. The environment represents zero-thickness flat states and does not model material mechanics or continuous collisions during folding.

Success requires geometric reasoning, state tracking, and planning together. Termination labels identify observed behavior but do not isolate the underlying cause of failure. Revisit and budget limits also constrain the search trajectories that can be observed. Generating new samples reduces reliance on a fixed public test set but cannot guarantee absence of training overlap.

Corpus replay checks share the generation engine and therefore test consistency rather than independent physical validity. Final comparisons use numerical tolerances. The shared terminal comparator does not explicitly match original-sheet region identities, as detailed in Appendix A.5; identity-aware validation and any re-scoring must be tied to the implementation used for the reported runs.

7 LICENCE AND AVAILABILITY

The corpus generator, the environment and verifier, both corpus checks, the state comparator and the evaluation harness are released under the **BSD 3-Clause** licence.

Code and data are released on publication; links are omitted here.

The artifacts carry no personal data, no human-subject data and no scraped content. Every sample is synthesised by folding a square, so no licensing question attaches to the geometry and no attribution is owed to any origami designer. Flat-Folder’s `examples/instagram/` set, the one external corpus this paper measures against, is not redistributed; the scope measurement of Section 6 was computed against it in place and is reported as a number.

8 USE OF LARGE LANGUAGE MODELS

Language models are evaluated as agents in the folding task. AI assistants were also used for implementation, literature-search assistance, and manuscript drafting and editing. Claude Code was used for coding assistance. AI-generated figure drafts served as composition references; dataset geometry and experimental results must be rendered from the corresponding artifacts. The authors are responsible for the manuscript and released implementation.

ACKNOWLEDGMENTS

Withheld for anonymous review.

REFERENCES

- Hugo A. Akitaya, Jun Mitani, Yoshihiro Kanamori, and Yukio Fukui. Generating folding sequences from crease patterns of flat-foldable origami. In *ACM SIGGRAPH Posters / Student Research Competition*, 2013.
- Hugo A. Akitaya, Erik D. Demaine, and Jason S. Ku. Computing flat-folded states. In *Origami⁸: Eighth International Meeting on Origami in Science, Mathematics and Education (OSME)*, 2024.
- Esther M. Arkin, Michael A. Bender, Erik D. Demaine, Martin L. Demaine, Joseph S. B. Mitchell, Saurabh Sethia, and Steven S. Skiena. When can you fold a map? *Computational Geometry*, 29(1):23–46, 2004.
- Marshall Bern and Barry Hayes. The complexity of flat origami. In *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 175–183, 1996.
- Creasy. Creasy, 2022. Open-source crease-pattern-to-sequence implementation, Java, GPL-3.0, unmaintained since 2022. <https://github.com/xkevio/Creasy>.
- R. Spencer, R. Yaari, R. Vemavarapu, J. Yang, S. Ngo, and U. Sharma. GamiBench: Evaluating spatial reasoning and 2D-to-3D planning capabilities of MLLMs with origami folding tasks. *arXiv preprint arXiv:2512.22207*, 2025. VERIFIED 2026-09-21: 186 regular and 186 impossible patterns, six viewpoints, three VQA tasks, VC and IFSR metrics.
- R. Xu, D. Lu, Z. Zhao, X. Tan, X. Wang, S. Yuan, J. Chen, and Y. Xu. OrigamiSpace: Benchmarking multimodal LLMs in multi-step spatial reasoning with mathematical constraints. In *Advances in Neural*

Information Processing Systems (NeurIPS), 2025. VERIFIED 2026-09-21: NeurIPS 2025 poster; also arXiv:2511.18450.

A STATE REPRESENTATION AND FOLDING CONSTRAINTS

A.1 FLAT-FOLDED STATES

A state represents the positions of paper regions in the plane, their orientations, and their layer ordering. Original-sheet coordinates identify each region and preserve its connectivity to other regions as folds change its current position. Overlapping regions can occupy the same planar area while remaining distinct layers. Their original-sheet identities therefore matter both when validating a move and when comparing the completed construction with the target. Layer ordering is also a central component of computational treatments of flat-folded states (Akitaya et al., 2024).

A.2 SUPPORTED ACTIONS

The simple-fold literature distinguishes operations by the layers they move (Arkin et al., 2004). Our all-layers action selects the paper on one side of a fold line throughout the stack. Our some-layers action restricts that selection to a contiguous run from the top or bottom. An action specifies the line in current folded coordinates, the moving side, the layer selection, and whether the moving paper folds over or under the stationary paper. The selected paper is reflected across the line, with the corresponding orientation and stack-order updates. In the implemented some-layers model, a top run folds over and a bottom run folds under.

These action models specify the permitted moves. They are distinct from the assistance conditions, which specify the observations, feedback, and tools available to the model. In particular, access to candidate enumeration changes the assistance provided without defining a new folding operation.

A.3 CONNECTIVITY AND TEARING

A move must preserve the connections between regions of the original sheet. If moving paper is attached to stationary paper along a segment away from the proposed hinge, the operation would separate connected paper and is rejected with a `would-tear` response. The check uses original-sheet connectivity together with the proposed motion; overlap in the current plane alone cannot establish whether two regions are attached. Rejection leaves the current state unchanged.

Tearing is one action constraint rather than a complete model of physical feasibility. The engine also checks the action parameters and compatibility with the target crease pattern. It represents zero-thickness flat states and does not test continuous collisions during the folding motion. Step-level legality and final target agreement are separate checks: a sequence of legal actions can still produce the wrong construction.

A.4 DEPTH AND SEARCH DIFFICULTY

The recorded sequence provides a reachable target, but need not be a shortest solution. Reference depth therefore describes generation rather than minimum planning horizon. Likewise, the number of legal moves and the probability that a random proposal is legal are different quantities: a larger proposal space can lower the latter without reducing the former. We use observed solver performance to assess difficulty rather than inferring it solely from either quantity or from worst-case complexity results for other folding variants.

A.5 COMPARISON IMPLEMENTATION

The engine compares creases in original-sheet coordinates after grouping and merging collinear intervals by assignment. Interval endpoints are matched within 2×10^{-6} ; coordinate canonicalization is part of preprocessing and does not make the comparison tolerance-free.

The shared terminal comparator first checks layer counts. It considers direct alignment and turnover, with turnover reversing layer order and face parity. Correspondences between vertices of the first layer propose global alignments; a candidate alignment is accepted only if every layer polygon and parity matches its target rank. Polygon matching uses a positional tolerance of 2×10^{-6} , short-edge cleanup at 10^{-7} , and a collinearity threshold of 10^{-9} .

The shared implementation represents current-plane polygons, parity, and rank. It does not explicitly compare original-sheet region identities. Any identity-aware offline comparator must be distinguished from this path, and the comparator version used for each reported result must be recorded. In particular, equal shapes at different original-sheet locations can require a stronger correspondence check than current-plane geometry alone.

A.6 ACTION REJECTION CONDITIONS

Table 10: Geometric and action constraints checked by the implementation. Rejected edits preserve the previous state.

Code	Condition
would-tear	A moving region and a stationary region share an original-sheet edge whose current position is not on the fold line. Reflecting one side would separate the join.
no-crease	A nonempty moving selection produces no crease segment in the paper. This condition is checked separately from tearing.
nothing-to-move	The selected half-plane contains no selected paper, or the requested partial selection contains no layers.
direction-impossible	The action requests folding a top run under the stack or a bottom run over it, contrary to the supported layer-selection rule.

The independent Flat-Folder wrapper uses layer-order constraints, including taco-taco, taco-tortilla, tortilla-tortilla, and transitivity constraints (Akitaya et al., 2024). It tests whether the pattern admits a flat-folded state, not whether the agent’s sequence reaches the supplied target. It is not invoked automatically for every proposed action.

B TOOLS, PROMPTS, AND RUN INSPECTION

B.1 TOOL SCHEMAS AND PROMPT

The following source listings document the available arguments and the folding prompt. Tool exposure varies by run configuration; the full schema listing is not a claim that every run exposed every tool.

```

"""Schemas supplied to the native Qwen3.5 tool-aware vision renderer."""
import json

def tool(name, description, properties=None, required=None):
    return {"type": "function", "function": {"name": name, "description": description,
        "parameters": {"type": "object", "properties": properties or {},
            "required": required or [], "additionalProperties": False}}}

TOOLS = [
    tool("add_fold", "Append a 180-degree fold. Choose angle_index OR angle_degrees. Partial top/bottom runs must not tea
        "angle_index": {"type": "integer", "enum": [0, 1, 2, 3]},
        "angle_degrees": {"type": "number", "description": "Line angle counterclockwise from +x; offset uses unit normal"},
        "selection_mode": {"type": "string", "enum": ["all", "top", "bottom"]},
        "layer_count": {"type": "integer", "minimum": 1, "description": "Required for top/bottom, omit for all."},
        "offset": {"type": "number"}, "move_positive": {"type": "boolean"},
        "over": {"type": "boolean"}}, [{"offset", "move_positive", "over"}],
    tool("remove_fold", "Remove 1-based fold; replay the rest transactionally.",
        {"step": {"type": "integer", "minimum": 1}}, [{"step"}],
    tool("go_to_step", "Keep the first N folds; zero clears the sequence.",
        {"step": {"type": "integer", "minimum": 0}}, [{"step"}],
    tool("restore_revision", "Restore a returned revision, including abandoned branches.",
        {"revision": {"type": "integer", "minimum": 0}}, [{"revision"}],
    tool("get_images", "Render top, two oblique, X-ray, and exploded-stack PNGs at a completed step.",
        {"step": {"type": "integer", "minimum": 0}}],
    tool("get_state", "Inspect the current sequence and bottom-to-top layer polygons."),
    tool("list_legal_folds", "List the folds that are legal in the current state and on-target for the CP. "
        "Read-only. Every listed action is accepted verbatim by add_fold while the state is unchanged.", {
        "max_results": {"type": "integer", "minimum": 1},
        "selection_filter": {"type": "string", "enum": ["any", "all", "top", "bottom"]},
        "include_rejected": {"type": "boolean"}}},
    tool("compare_to_target", "Compare the current stack with the target folded state. Read-only. "
        "Detail is fixed by the run's configured tier, not by you. Uses the target state only; "
        "the reference fold sequence is never consulted."),
    tool("finish", "End this episode and evaluate the current sequence."),
]

# The Codex loop exposes only a subset in its baseline condition; the enumerator changes
# what the benchmark measures, so it must be requested explicitly. See TODO.md.
ENUMERATION_TOOLS = ("list_legal_folds",)
COMPARE_TOOLS = ("compare_to_target",)

```

```

def _whole_stack_only(tools):
    """Strip partial folds from the schemas, so nothing offers what ToolSession will refuse.

    Two edits, and both are needed. add_fold loses selection_mode and layer_count, or the
    model can request a run the simulator then rejects; list_legal_folds loses
    selection_filter, whose remaining values would all be lies once the run forces "all".
    """
    out = []
    for t in tools:
        name = t["function"]["name"]
        if name not in ("add_fold", "list_legal_folds"):
            out.append(t)
            continue
        copy = json.loads(json.dumps(t))
        props = copy["function"]["parameters"]["properties"]
        for key in ("selection_mode", "layer_count", "selection_filter"):
            props.pop(key, None)
        if name == "add_fold":
            copy["function"]["description"] = (
                "Append a 180-degree fold of the whole stack. Choose angle_index OR angle_degrees.")
        out.append(copy)
    return out

def tools_for(mode, compare_tier=0, action_space="any"):
    """The action set for one experimental condition.

    compare_to_target appears only when a tier is configured, so the default arms are
    byte-identical to every run recorded before the comparison tool existed. action_space
    defaults to "any" for the same reason: nothing changes unless a run asks for it.

    action_space is a property of the corpus, not a preference. On release/all-layers every
    one of the 4180 reference folds is whole-stack, so partial folds cost 3-87x in search and
    buy nothing; on some-verified-d3 and some-generated-d6 they are what 14% and 34% of the
    samples require. Whichever is chosen must be given to the search arm too, or the
    comparison measures the setting rather than the solver.
    """
    if mode == "base":
        chosen = [t for t in TOOLS if t["function"]["name"] not in ENUMERATION_TOOLS]
    elif mode == "legal-folds":
        chosen = list(TOOLS)
    else:
        raise ValueError(f"Unknown tool mode: {mode}")
    if not compare_tier:
        chosen = [t for t in chosen if t["function"]["name"] not in COMPARE_TOOLS]
    if action_space == "all-layers":
        chosen = _whole_stack_only(chosen)
    elif action_space != "any":
        raise ValueError(f"Unknown action space: {action_space}")
    return chosen

```

Recover a folding sequence from the supplied crease pattern (CP) and final folded state. The sequence starts empty. Use one tool call at a time, then inspect its result before proposing the next fold. Do not return an entire sequence in prose. The environment writes your accepted actions into seq.json.

You receive only geometry and images of the CP, final target, and current state. No reference folding sequence is supplied. All folds are flat 180-degree simple folds. Select all layers (default), or a contiguous run from the top or bottom.

Coordinates are x right, y up. Fold axes are in CURRENT folded coordinates:

```

| angle_index | fold line | positive side |
| --- | --- | --- |
| 0 | y = offset | y > offset |
| 1 | y = x + offset | y - x > offset |
| 2 | x = offset | x > offset |
| 3 | x + y = offset | x + y > offset |

```

For arbitrary line angles, supply angle_degrees instead of angle_index, never both. With theta counterclockwise from +x, the line is $-\sin(\theta)x + \cos(\theta)y = \text{offset}$; offset is signed perpendicular distance. move_positive selects the greater-than side. The folding motion remains 180 degrees. selection_mode is all (default), top, or bottom. Top/bottom requires layer_count;

omit `layer_count` for all. Top must fold over; bottom must fold under. The engine tracks original-sheet connectivity. A partial fold returns would-tear if selected paper is attached to stationary paper away from the proposed hinge. For example, selecting three of four layers may tear a paired flap. This check runs before target-CP checking and leaves state unchanged on rejection. Inspect `sheet_polygon` coordinates and exploded views when choosing a connected run.

`move_positive` selects which half-plane moves. `over=true` puts the reflected moving stack above the stationary stack; `false` puts it below. A flipped stack reverses its internal order. Offset is not a normalized distance for diagonal axes. The tool rejects extra creases or incompatible mountain/valley assignments and leaves the state unchanged on error. This is a zero-thickness geometric engine; it does not check continuous collisions during motion.

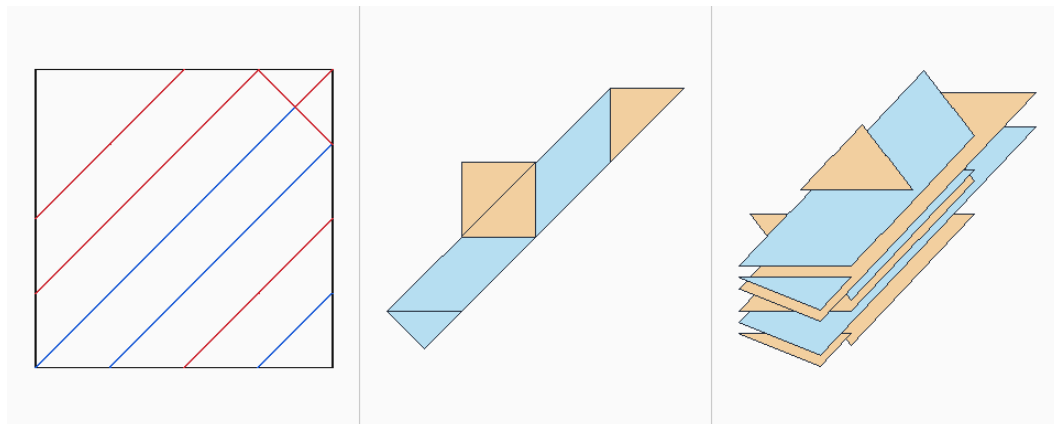
`add_fold` appends ONE fold. `remove_fold` removes a 1-based fold and replays the remaining actions; an invalid replay leaves the sequence unchanged. `go_to_step` keeps the first N folds (0 empties the sequence). `restore_revision` restores a previous tool state, even after changing branches; revision 0 is the empty sheet. These editing operations create new revision IDs. `get_state` returns your current sequence and geometry. `get_images` returns PNG views of any completed step in the current branch without changing it. Use the returned revision IDs when backtracking.

Images: CP red = mountain, blue = valley, dark boundary = paper edge. Top and two oblique views show paper shape with a small illustrative layer separation. Oblique views look down from opposite directions, never exactly edge-on. X-ray is a top projection: more overlapping layers produce darker pixels. Each view is fitted independently; use geometry for coordinates and scale, not pixel size. Exploded views separate layers upward for inspection: layer 1 is the bottom, and `p0/p1` labels give face parity. Separation is illustrative, not paper thickness. Initial images include the target's exploded view; feedback images include the current state's exploded view after edits.

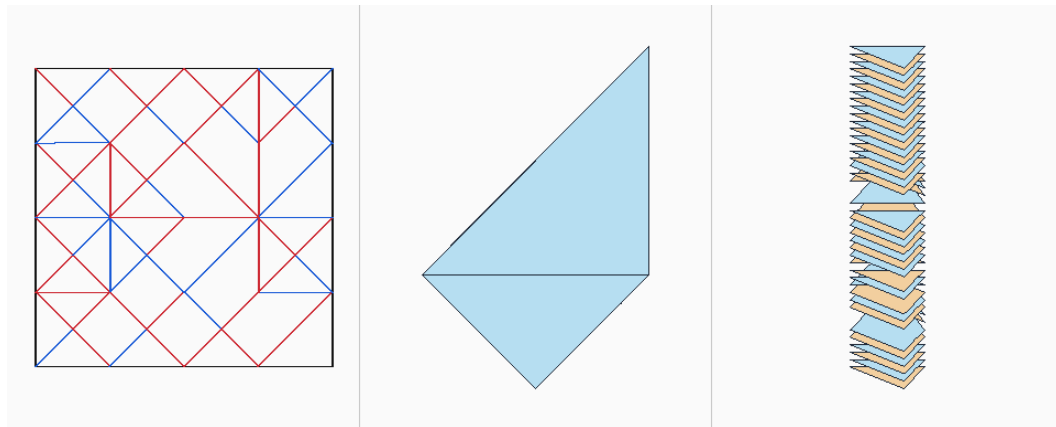
Try to reproduce both the CP and the supplied target, including face orientation and layer order. Terminal matching accepts any global translation and any rotation angle, including 30 degrees. It also accepts turning the whole model upside down: reflect the geometry, reverse the bottom-to-top stack, and flip every face parity. These operations can be combined. One common rigid transformation must match all layers; arbitrary layer rearrangements, scaling, or independently moving layers do not count. CP matching still uses the supplied original-sheet coordinates and mountain/valley assignments. Call finish when you are done. This ends the episode and evaluates current sequence; it does not automatically claim success.

B.2 DATASET GALLERY AND REFERENCE SEQUENCES

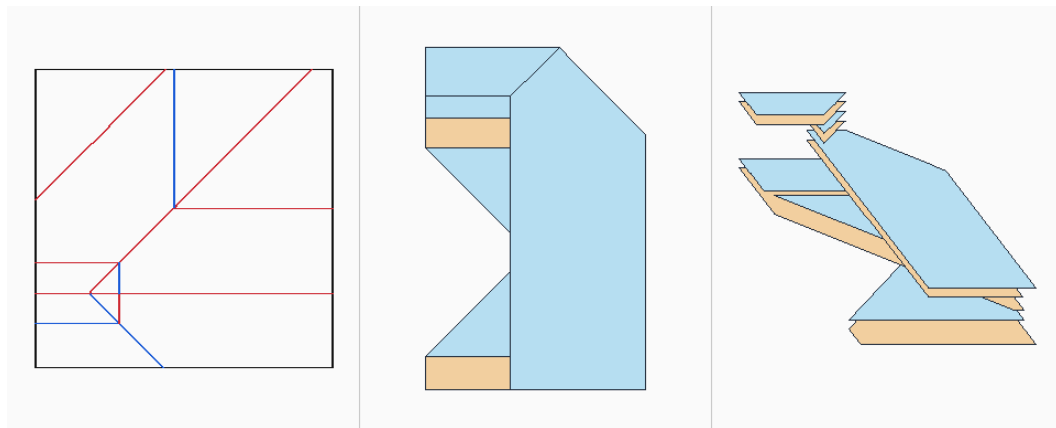
Each rendered panel contains the crease pattern, top view, and an oblique view with illustrative layer separation. Layer separation is a visualization aid, not simulated paper thickness. `all-layers/easy-0001` (all-layers, 4 reference folds)



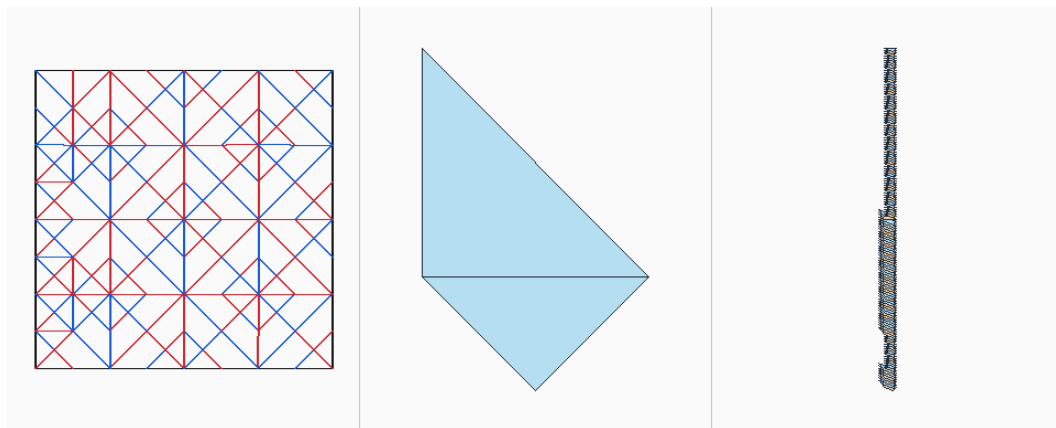
all-layers/mid-0004 (all-layers, 11 reference folds)



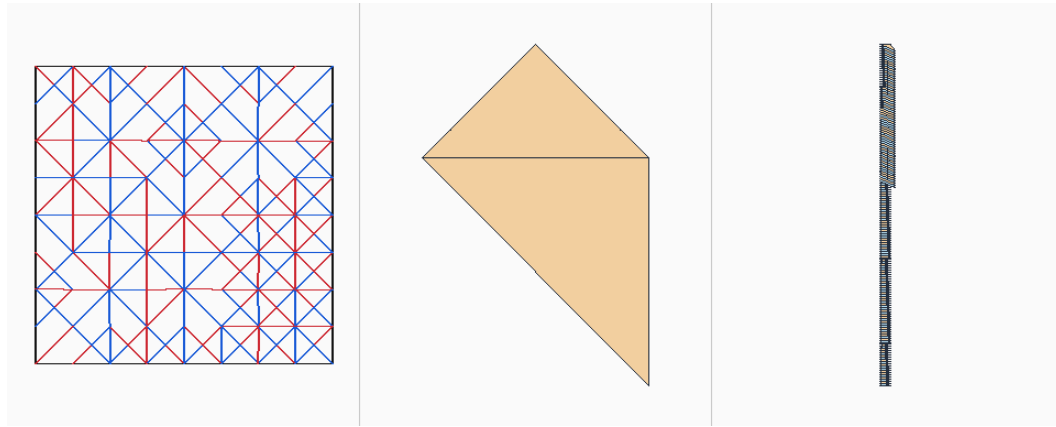
some-generated-d5/layers-0001 (some-layers, 5 reference folds)



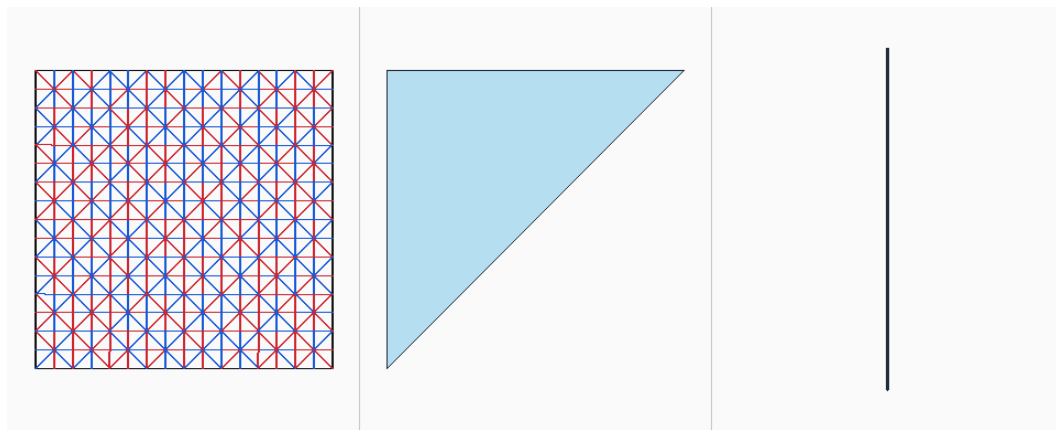
all-layers/hard-0004 (all-layers, 15 reference folds)



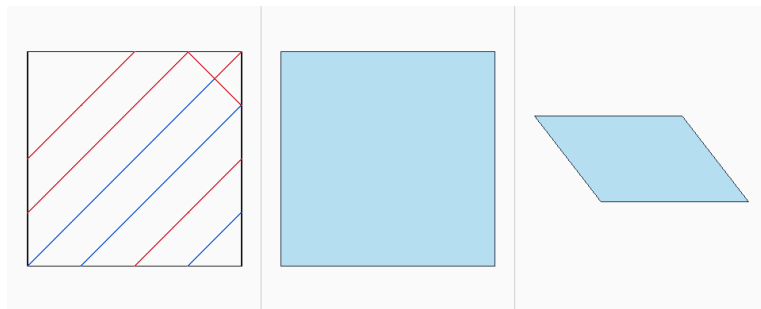
all-layers/hard-0005 (all-layers, 15 reference folds)



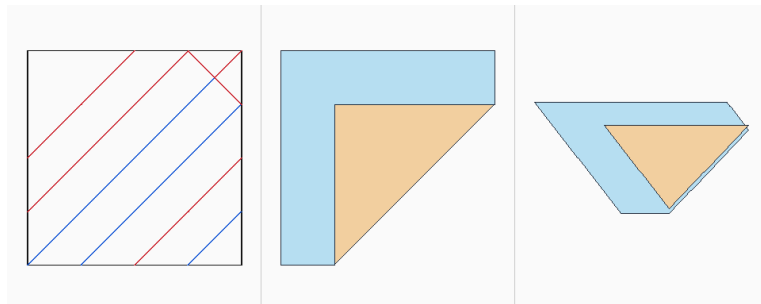
all-layers/hard-0006 (all-layers, 15 reference folds)



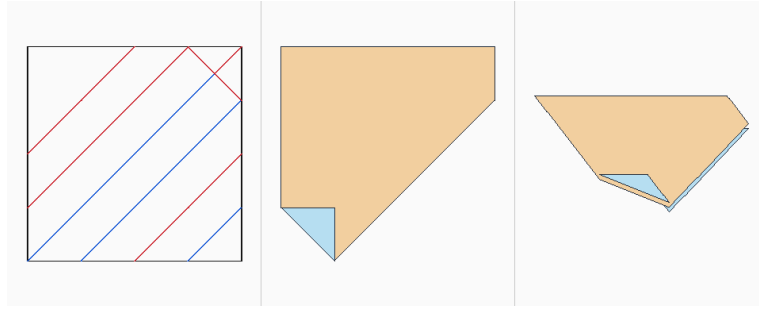
all-layers/easy-0001 (all-layers, 4 reference folds) Stored state 0



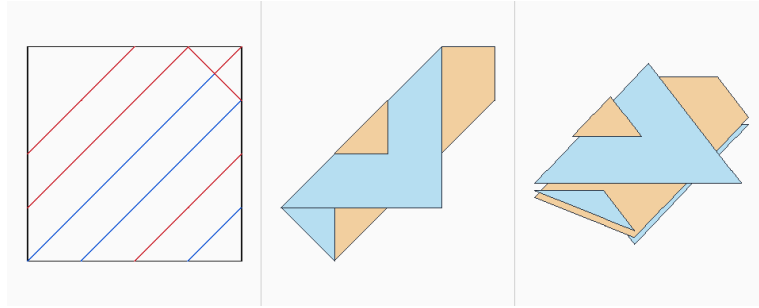
Stored state 1



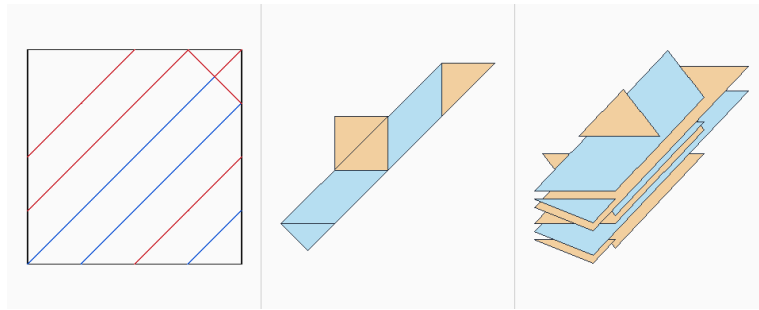
Stored state 2



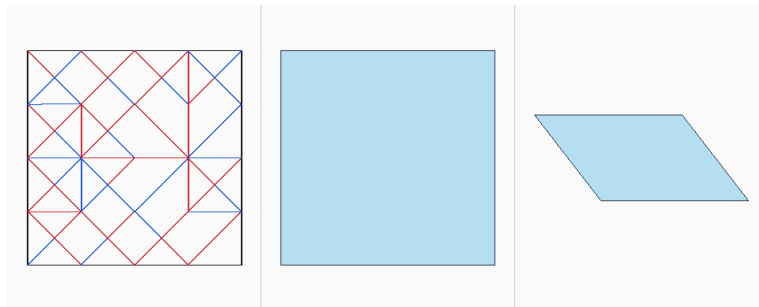
Stored state 3



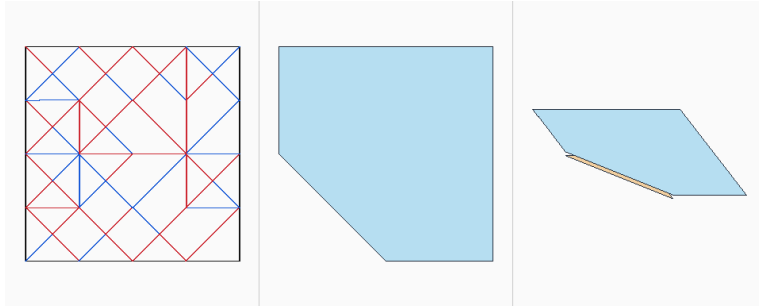
Stored state 4



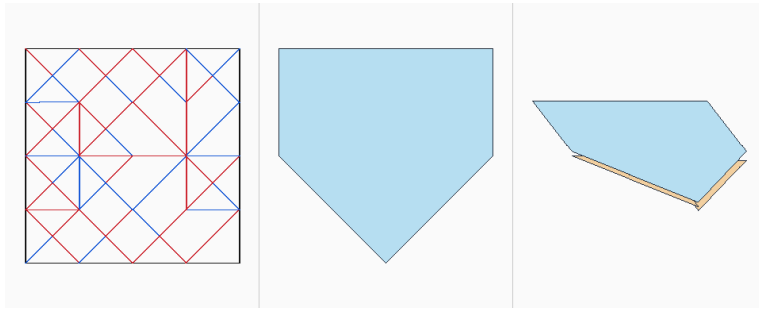
all-layers/mid-0004 (all-layers, 11 reference folds) Stored state 0



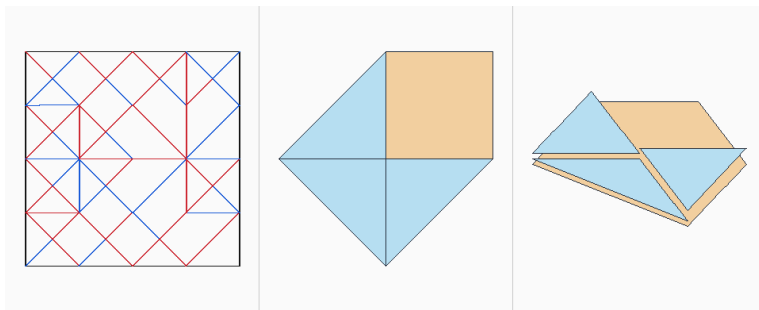
Stored state 1



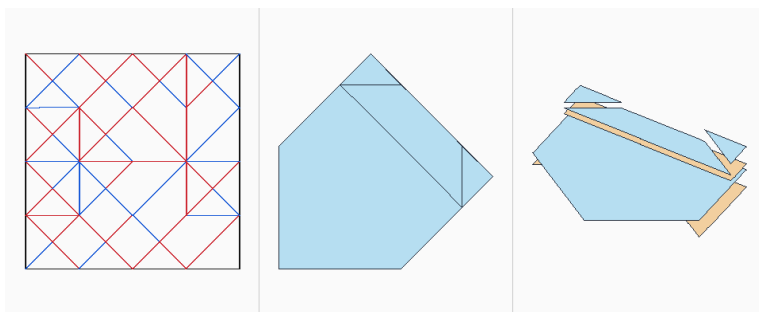
Stored state 2



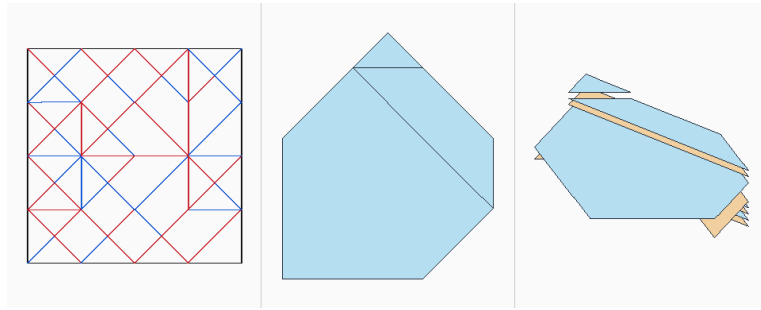
Stored state 3



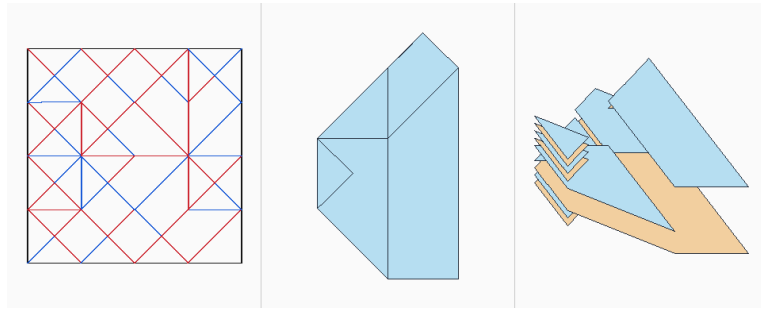
Stored state 4



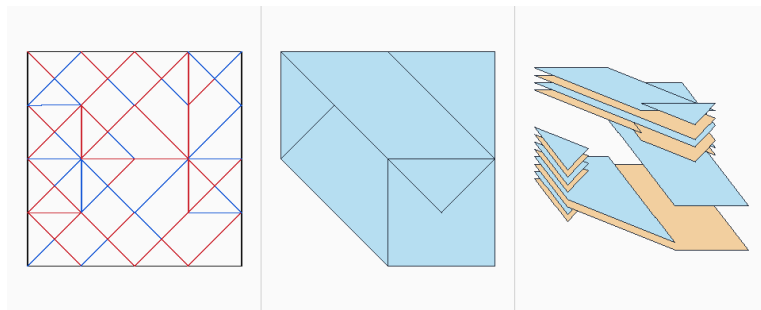
Stored state 5



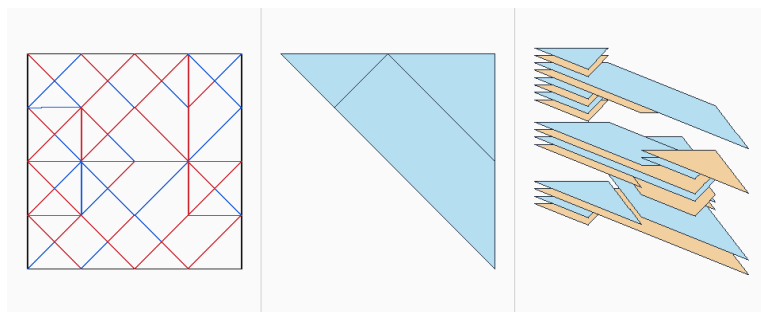
Stored state 6



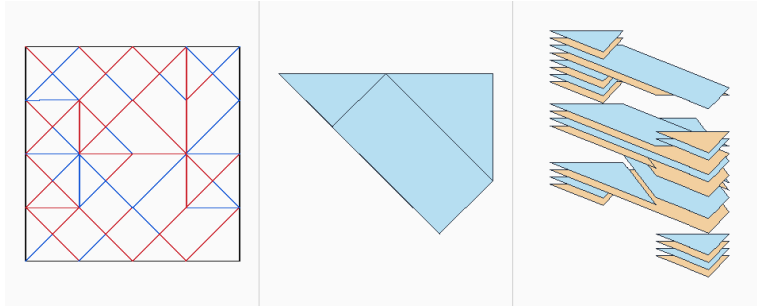
Stored state 7



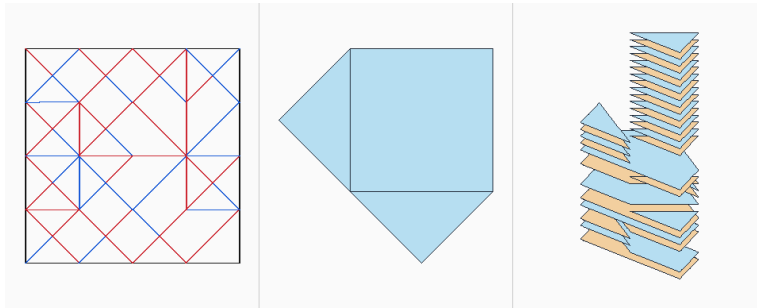
Stored state 8



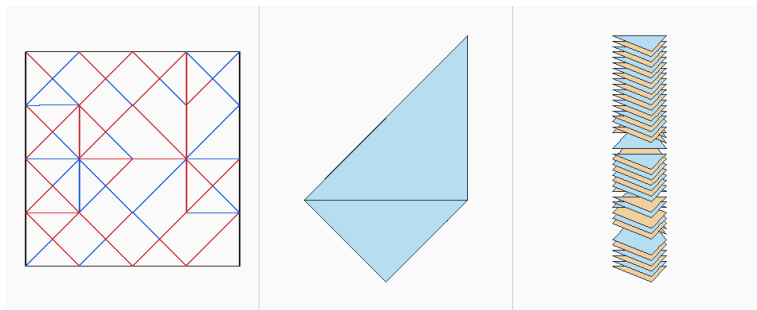
Stored state 9



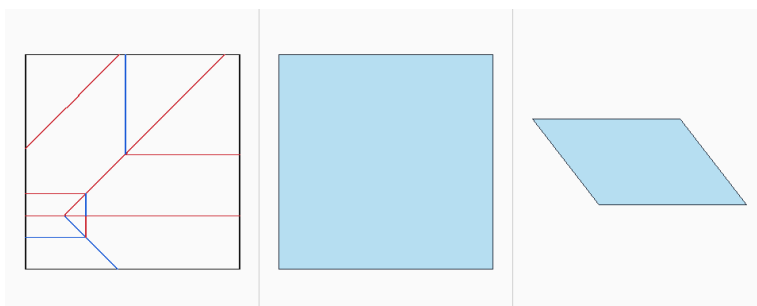
Stored state 10



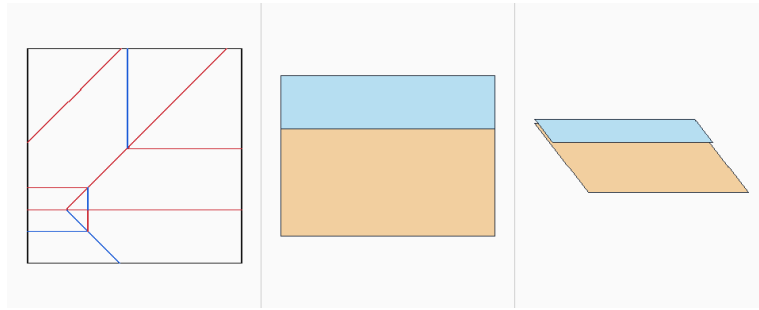
Stored state 11



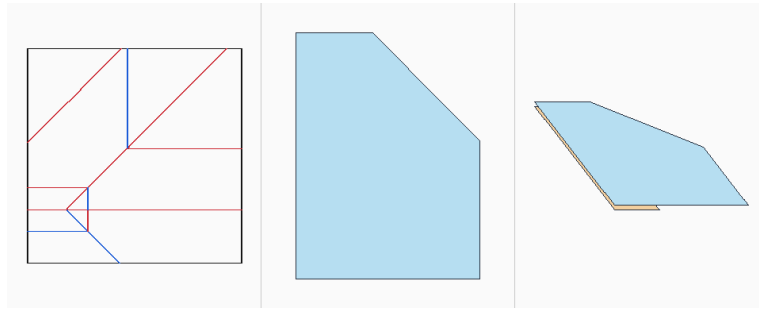
some-generated-d5/layers-0001 (some-layers, 5 reference folds) Stored state 0



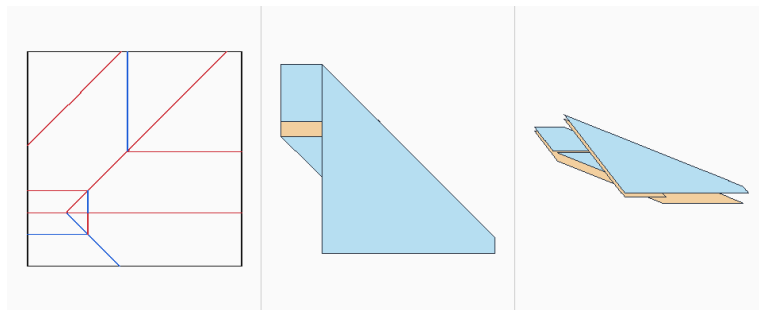
Stored state 1



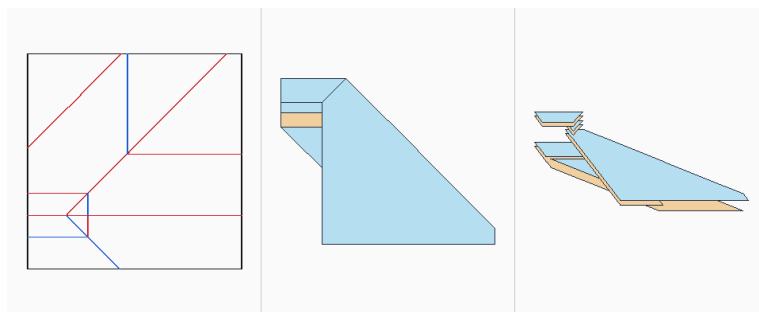
Stored state 2



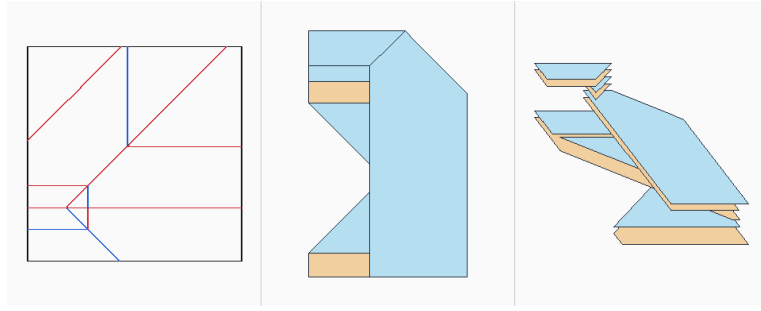
Stored state 3



Stored state 4



Stored state 5



B.3 TRACE-ANALYSIS INTERFACE

We provide a trace-inspection interface for browsing saved runs, replaying recorded folding trajectories, and examining model actions alongside tool feedback and rendered states. This supports qualitative analysis of successful construction, repeated exploration, and unsuccessful termination. The interface presents recorded artifacts; it does not establish the model’s internal reasoning process. A command-line trace viewer additionally supports filtering by task, trial, session, role, and event type.

B.4 COMPARATOR VALIDATION

Implementation details and current representation limits are described in Appendix A.5. The repository contains comparison and verification tests; executed test results and the exact comparator revision must accompany any reported validation.

B.5 TOOL LOADING AND CONFIGURATION

The schema builder selects the tools exposed in each run. Basic mode excludes candidate enumeration; filtered mode includes it. Target comparison is added only when a comparison level is configured. For an all-layers action model, partial-layer selection arguments are removed from the exposed action schemas. The run configuration therefore records tool mode, comparison level, and action model separately.